Sous-système de travaux par lots BATCH

Guide du programmeur

BATCH-PG-0101

BATCH version 1.0.1 *** 4-AOÛ-2026

Table des matières

Préface **3**
Public visé 3
Structure du document 4
Conventions 5

Chapitre 1 **6**

Le concept **7**
1.1 Ce qu'est le sous-système 7
1.2 Les décisions, et pourquoi 8
1.3 Ce qui n'est délibérément pas fait 9

Chapitre 2 **10**

L'architecture **11**
2.1 Les deux images 11
2.2 La file 12
2.3 Le trajet d'une requête 13
2.4 La boucle principale 14

Chapitre 3 **15**

Les modules **16**
3.1 Carte 16
3.2 batchdef.h 17
3.3 batchmsg.h et batchmsg.c 18
3.4 batchipc.h 19
3.5 util.c 20
3.6 batchd.c 21
3.7 batch.c 22
3.8 Le paquet StarLet 23

Chapitre 4 **24**

Les structures de données **25**
4.1 BATCH$JOB 25
4.2 BATCHD$JOBENT et BATCHD$CONN 26
4.3 Le protocole 27
4.4 Le journal de la file 28

Chapitre 5 **29**

Les conventions **30**
5.1 La nomenclature 31
5.2 La mise en forme 32
5.3 La convention d'appel 33
5.4 Les diagnostics 34

5.5 La mise en forme des sorties 35

Chapitre 6 **36**

Faire une modification **37**
6.1 Ajouter un qualificatif 37
6.2 Ajouter une condition 38
6.3 Ajouter une commande 39
6.4 Modifier le descripteur de travail 40
6.5 Ajouter une vérification au test 41

Chapitre 7 **42**

Le système de construction **43**
7.1 Disposition 43
7.2 Les options de compilation 44
7.3 Deux maladresses 45

Chapitre 8 **46**

Les tests **47**
8.1 Ce qu'il y a 47
8.2 Ce que le test de bon fonctionnement garde 48

Annexe A **49**

Limitations connues **50**

Préface

Public visé

Ce manuel s’adresse au programmeur appelé à modifier le sous-système : ajouter un qualificatif, un message ou une commande, corriger un défaut, ou s’assurer qu’un changement est sans danger. Il suppose le C, POSIX et une familiarité de lecture avec CMake. Il ne suppose pas OpenVMS, encore que le lecteur qui le connaît reconnaîtra pourquoi une grande part de la conception se présente ainsi.

Le *Guide de l’utilisateur* décrit ce que le sous-système fait. Le présent manuel décrit comment il est construit ; ni l’un ni l’autre ne répète son voisin.

Structure du document

Partie	Contenu
Chapitre 1	Ce qu’est le sous-système et les décisions sur lesquelles il repose
Chapitre 2	L’architecture : les deux images, la file, le trajet d’une requête
Chapitre 3	Les modules, un à un, et ce à quoi sert chaque routine
Chapitre 4	Les structures de données et les deux formats qui survivent à un processus
Chapitre 5	Les conventions que suivent les sources, et pourquoi
Chapitre 6	Comment faire les modifications qui reviennent le plus souvent
Chapitre 7	Le système de construction
Chapitre 8	Les tests et ce qu’ils gardent
Annexe A	Les limitations connues et où chacune se trouve

Conventions

Les numéros de ligne ne sont pas cités : ils vieillissent. On nomme la routine, et chaque routine porte en tête un bloc `FUNCTIONAL DESCRIPTION` qui dit ce qu'elle fait, ce qu'elle prend et ce qu'elle rend.

Chapitre 1

Le concept

1.1 Ce qu’est le sous-système

BATCH exécute des scripts d’interpréteur en travaux par lots : un script est placé dans une file d’attente et exécuté plus tard, sans terminal, sous la conduite d’un contrôleur de travaux résident. C’est l’analogue fonctionnel de la file de travaux par lots d’OpenVMS — `SUBMIT`, `SHOW QUEUE`, `DELETE /ENTRY` — et il suit ce modèle délibérément, plutôt que celui de `cron` ou d’un superviseur de services.

Trois propriétés découlent du modèle et façonnent tout le reste :

- **Un travail est identifié avant d’exister.** La soumission attribue un numéro d’entrée et rend la main ; le processus est créé plus tard, si et quand le contrôleur en décide. C’est à l’entrée, non au processus, que l’utilisateur se tient.
- **La file a des états.** Un travail peut attendre une place d’exécution ou une heure fixée. C’est le contrôleur, non l’appelant, qui décide du moment.
- **Un travail qui échoue demeure.** Son entrée et son journal survivent afin qu’on puisse les examiner. Un travail qui réussit quitte la file. C’est ce qui rend `SHOW` utile sans qualificatif : ce qui reste est ce qui s’exécute et ce qui a mal tourné.

1.2 Les décisions, et pourquoi

Deux images, non une. `batchd` possède la file ; `batch` est un client qui ne possède rien. La solution inverse — un utilitaire `setuid` manipulant un fichier partagé — remettrait les invariants de la file entre les mains de tout appelant.

L'identité vient du noyau. Le contrôleur prend l'identité d'un client dans `SO_PEERCRED`, jamais dans le contenu d'une requête. Un client peut donc mentir sur tout sauf sur qui il est, et tout ce qui importe est vérifié contre cela.

Un contrôleur, un seul fil d'exécution. La file n'est pas verrouillée parce que rien d'autre n'y touche. C'est une contrainte réelle sur ce qu'on peut y ajouter : ce qui bloque la boucle principale arrête tout le sous-système, et c'est pourquoi les connexions sont servies sans blocage (section 2.4).

Descripteur de taille fixe, sans sérialisation. `BATCH$JOB` est plat et de taille fixe ; il est écrit dans le journal et posé sur le socket tel quel. Le prix en est que l'on déplace des enregistrements presque vides ; le gain, qu'il n'y a pas de code de sérialisation à manquer, et que le journal se lit avec `od`. La section 4.4 dit ce qui en découle pour la compatibilité.

Des valeurs de condition, non des codes d'erreur. Toute routine rend une valeur de condition à la manière d'OpenVMS : installation, numéro de message, sévérité. Le bit de poids faible est l'indicateur de réussite ; une issue s'éprouve donc sans égard à la condition qu'elle porte, et la cause se distingue par `$MATCH_COND` plutôt qu'en comparant du texte.

La bibliothèque d'autrui. La journalisation, les messages, l'interpréteur de commandes, les primitives de file et le moteur FAO viennent du paquet StarLet. Le sous-système n'apporte aucun code d'usage général qui lui soit propre, et là où la bibliothèque s'est trouvée en défaut, la correction est allée dans la bibliothèque.

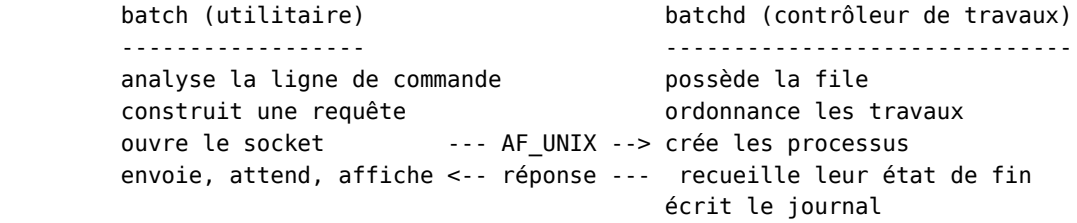
1.3 Ce qui n’est délibérément pas fait

- **Le sous-système n’arrête pas un travail qu’il ne conduit plus.** Un travail est une session à lui. Quand le contrôleur s’arrête, les travaux en cours continuent ; au démarrage suivant ils sont marqués orphelins, ou remis en file s’ils portaient `/RESTART`. `/RESTART` promet qu’un travail sera exécuté, non qu’il le sera une seule fois.
- **Le sous-système n’a pas de fichier de configuration.** Les réglages viennent de la ligne de commande du contrôleur et de la construction. Il n’y a pas d’état à analyser, ni à laisser se désaccorder.
- **Le sous-système ne gère pas les utilisateurs.** Ni comptes, ni quotas, ni listes d’accès. Un travail s’exécute sous le nom de qui l’a soumis ; l’accès à une entrée se décide par la seule propriété.

Chapitre 2

L’architecture

2.1 Les deux images



`batch` ne garde aucun état d’un appel à l’autre. Tout ce qui survit à une commande est dans le contrôleur et dans les deux fichiers décrits à la section 4.4.

2.2 La file

La file est une liste doublement chaînée de `BATCHD$JOBENT`, tenue dans une `__QUEUE` de la bibliothèque. Chaque entrée enveloppe un descripteur `BATCH$JOB`. Une entrée est allouée quand un travail est mis en file et libérée quand elle en sort ; le contrôleur tient donc ce que ses travaux demandent, et non une table dimensionnée pour le pire cas.

Le parcours se fait par les chaînages directement — la bibliothèque offre le retrait par l’une ou l’autre extrémité, mais pas d’itérateur — et deux macros de `batchd.c` le disent sans détour :

```
#define $JOBQUE_FIRST()     ((BATCHD$JOBENT *) g_jobque.head)
#define $JOBQUE_NEXT(ent)   ((BATCHD$JOBENT *) (ent)->je_links.right)
```

`BATCH_K_MAXJOB` est un plafond, non une capacité : il existe pour qu’un soumetteur emballé ne puisse épuiser la mémoire, et la condition qu’il lève, `BATCH$_NOSLOT`, est plus utile au soumetteur qu’une allocation manquée ne le serait.

Les états par lesquels passe un travail sont énoncés au *Guide de l’utilisateur*, section 1.5. Dans les sources ce sont les constantes `JOB$K_*` de `batchdef.h`. La valeur zéro n’est attribuée à rien : une entrée de la file a toujours un état, et une entrée qui n’en a pas n’existe pas.

2.3 Le trajet d’une requête

```
batch                     batchd
-----                     ------
connect ----------------> s_batchd$conn_accept
                            SO_PEERCRED, O_NONBLOCK, mise en g_connque
envoi de l'en-tête -----> s_batchd$conn_read     état CONN$K_RDHDR
envoi de la charge -----> s_batchd$conn_read     état CONN$K_RDPLD
                          s_batchd$conn_dispatch
                            s_batchd$do_submit / do_delete / do_show / release
                            réponse assemblée en mémoire
réception <-------------- s_batchd$conn_write    état CONN$K_WRRSP
                          s_batchd$conn_close
```

La réponse est assemblée en entier avant que rien n’en soit transmis. C’est ce qui empêche la file de rester ouverte pendant un transfert vers un lecteur lent : ce que le client reçoit est la file telle qu’elle était quand il a demandé.

2.4 La boucle principale

`main`, dans `batchd.c`, est une boucle, et tout ce que fait le contrôleur s’y passe :

```
recueillir les travaux terminés     s_batchd$reap
ordonnancer ce qui peut s'exécuter  s_batchd$schedule
écrire le journal si l'un des deux a changé quelque chose
rembobiner le journal du contrôleur s'il a grossi
balayer les journaux de travaux, une fois l'heure
bâtir l'ensemble surveillé : socket d'écoute, tube interne, chaque connexion ouverte
poll
vider le tube interne
avancer d'un pas chaque connexion prête
accepter une nouvelle connexion
réordonnancer et journaliser si une requête a changé la file
```

Deux propriétés portent l’édifice.

Rien ne bloque. Tout descripteur est non bloquant et n’est avancé que lorsque `poll` le dit prêt. Un client qui se connecte et ne dit rien coûte un descripteur et est abandonné au bout de `BATCHD$K_CONNTMO` secondes.

Les signaux arrivent par un tube interne. Le gestionnaire écrit un octet et rend la main ; tout le reste se passe dans le contexte principal, où le choix des routines n’est pas restreint. `SIGCHLD` n’entre donc pas en concurrence avec la file.

La borne supérieure de l’attente, `BATCHD$K_POLLMS`, existe pour `/AFTER` : l’arrivée d’une heure fixée ne produit aucun événement, et la boucle doit donc se réveiller pour regarder.

Chapitre 3

Les modules

3.1 Carte

Module	Objet
`batchdef.h`	Les états, le descripteur `BATCH$JOB`, les capacités, les prototypes des routines d'usage général
`batchmsg.h`	Les valeurs de condition de l'installation, les tests d'issue
`batchmsg.c`	La table des messages et son enregistrement
`batchipc.h`	Le protocole entre les deux images
`batchjrn.h`	Les étiquettes du journal de la file et les règles qui les régissent
`util.c`	Routines d'usage général : sortie, temps, chemins, transfert
`batchd.c`	Le contrôleur de travaux
`batch.c`	L'utilitaire de ligne de commande
`batchvers.h.in`	L'identification, remplie par CMake

L'inclusion va dans un seul sens : `batchdef.h` tire `batchvers.h`, `batchmsg.h` et `starlet.h`, et `batchmsg.h` tire `utility_routines.h`. Rien n'inclut `batchd.c` ni `batch.c` ; ils ne partagent que `util.c` et les en-têtes.

L'ordre importe en un endroit. `utility_routines.h` déclare les codes de sévérité en énumération, et `starlet.h` porte des macros de repli des mêmes noms sous `#ifndef`. La bibliothèque doit donc être incluse la première, ce dont `batchdef.h` se charge. L'ordre inverse change l'énumération en une affectation à une constante.

3.2 batchdef.h

Les définitions communes aux deux images. Trois choses y vivent, et nulle part ailleurs :

- les capacités — `BATCH_K_MAXJOB`, `BATCH_K_NPARM`, `BATCH_K_PATHSZ` et les autres ;
- les états `JOB$K_*` et les conditions de conservation `RETAIN$K_*` ;
- `BATCH$JOB`, le descripteur, décrit à la section 4.1.

Il énonce aussi la convention d'appel que tout le sous-système suit, et que reprend la section 5.3.

3.3 batchmsg.h et batchmsg.c

`batchmsg.h` définit le numéro d'installation et une valeur de condition par issue distinguable. Les numéros de message sont répartis par domaines, avec des trous entre eux, afin qu'ajouter une condition ne déplace pas les numéros déjà attribués :

```
  1 ..   9   généralités          50 ..  69   fichiers et journal
 10 ..  29   usage général        70 ..  89   communication
 30 ..  49   file et travaux      90 .. 109   ligne de commande
                                 110 .. 129   création de processus et contrôleur
```

`batchmsg.c` porte la table elle-même, un tableau d'`EMSG_RECORD`, et `msg$init` qui la remet à `__util$inimsg`. Les deux images appellent `msg$init` avant d'émettre quoi que ce soit. Trois contraintes pèsent sur la table :

- elle n'est **pas** `const` — la bibliothèque la trie sur place ;
- un texte de message est écrit pour le moteur **FAO** — `!AZ`, `!SL`, `!UL`, `!SQ` —, `__util$putmsg_fao` le mettant en forme par `__util$faol`. Le moteur ne connaît pas les conversions de printf et les imprime telles quelles. Le nombre de directives du texte doit égaler le nombre d'arguments de chaque point d'appel, ce que le compilateur ne relève pas ;
- l'enveloppe `%FAC-S-IDENT` n'est **pas** dans le texte : la bibliothèque la tire du champ `facname` du descripteur, et un texte qui porterait la sienne serait encadré deux fois.

Deux conditions ne peuvent partager un numéro. Un message se retrouve par lui : une collision remet donc les arguments d'un message au texte d'un autre, et un texte qui attend une chaîne là où un nombre a été passé emporte le processus. Il en a coûté un plantage de `SYNC` pour l'apprendre ; une vérification du test le garde désormais.

3.4 batchipc.h

Le protocole. Une trame est un `BATCH$IPCHDR` fixe suivi de `mh_len` octets de charge utile. L'ordre des octets n'est pas converti : les deux extrémités sont sur un même nœud et viennent d'une même construction. Les codes de fonction sont les `IPC$K_*` et les indicateurs de sélection de `SHOW` les `SHOW$M_*`.

3.5 util.c

Les routines qu'emploient les deux images. Aucune ne sait rien des travaux.

Routine	Objet
`util$put`	Met une ligne en forme par FAO et l'écrit sur la sortie standard. Toute ligne que l'utilitaire montre passe par là.
`util$mkfspec`	Construit une spécification de fichier par FAO, en refusant de tronquer. Les paramètres arrivent en tableau, une liste variable ne pouvant être transmise à `__util$faol`.
`util$fmt_time`	Rend une heure sous la forme `jj-MMM-aaaa hh:mm:ss` par `__util$asctim`, à l'horloge du lecteur.
`util$parse_time`	Analyse la grammaire de `/AFTER`. Adaptateur de syntaxe au-dessus de `__util$bintim` : il réécrit la spécification dans la forme que la bibliothèque accepte et lui laisse l'arithmétique.
`util$tzoffset`, `util$local2utc`	La conversion entre l'horloge du lecteur et UTC, prise à l'instant considéré et non à l'instant présent.
`util$parse_cpulist`, `util$fmt_cpulist`	La liste de `/CPU`, dans la notation de `taskset(1)`.
`util$job_encode`, `util$job_decode`	Un descripteur et son enregistrement de journal, décrits à la section 4.4.
`util$mkpath`	`mkdir -p`.
`util$page`	Met la sortie en pages là où il y a un terminal pour la retenir.
`util$state_name`	Mnémonique d'un état de travail.
`util$read_exact`, `util$write_exact`	Transfert bloquant d'un nombre exact d'octets. Employés par l'utilitaire et pour le journal — **non** par le contrôleur pour les connexions clientes, qui ne doivent pas bloquer.

3.6 batchd.c

Le contrôleur. Ses routines se rangent en cinq groupes.

La file. `s_batchd$job_alloc` et `s_batchd$job_free` sont les seules routines qui créent et détruisent des entrées ; `job_free` retire aussi le fichier journal, sauf si `/KEEP` a été donné ou si le journal a été nommé par le client. `s_batchd$job_by_entry` et `s_batchd$job_by_pid` retrouvent une entrée ; `s_batchd$count_executing` compte les places occupées.

Le journal. `s_batchd$journal_save` écrit toute la file dans un fichier temporaire et le renomme par-dessus la copie précédente, ce qui rend l'écriture atomique. `s_batchd$journal_load` le relit et décide du sort des travaux qui s'exécutaient quand le contrôleur s'est arrêté.

Les travaux. `s_batchd$job_start` crée le processus : session, signaux, affinité, identité, répertoire de travail, descripteurs, environnement, `execv`. `s_batchd$schedule` choisit ce qui s'exécute et libère ce qui a assez attendu. `s_batchd$reap` recueille l'état de fin des travaux terminés et décide si l'entrée demeure.

Les requêtes. `s_batchd$do_submit`, `s_batchd$do_delete`, `s_batchd$do_show`, `s_batchd$do_mgrstat` et la branche `IPC$K_RELEASE` de `s_batchd$conn_dispatch`. `s_batchd$job_selected` énonce en un seul endroit la règle de sélection de `SHOW`, car la file est parcourue deux fois — pour compter et pour transmettre — et les deux passages doivent s'accorder.

Les connexions. `s_batchd$conn_accept`, `conn_read`, `conn_dispatch`, `conn_write`, `conn_close` et l'état qu'elles tiennent dans `BATCHD$CONN`. `s_batchd$listen` crée le socket d'écoute ; `s_batchd$log_sweep` débarrasse le répertoire des journaux de ce qui n'appartient à aucune entrée.

La seule routine qui ne soit pas appelée depuis le contexte principal est `s_batchd$sighandler` : elle écrit un octet dans le tube interne et rend la main, et ne fait rien d'autre.

Le processus créé est un autre monde. Tout ce qui suit `fork` dans `s_batchd$job_start` s'exécute alors que les flux standard sont sur le point de devenir le journal du travail ; rien n'y doit donc émettre de message, qui atterrirait dans ce journal. Un échec y sort par `BATCHD$K_EXECFAIL`, que le recueilleur relève comme un état de fin ordinaire.

3.7 batch.c

L'utilitaire. Il analyse, envoie et affiche.

L'analyse est la partie où il y a une décision. En DCL un qualificatif commence par une barre oblique ; sous Unix un chemin absolu aussi. Les services CLI de la bibliothèque ne font pas cette distinction et, de plus, prennent un paramètre déclaré en tête du vecteur sans condition. Deux choses sont donc faites ici :

- `s_batch$is_qual` classe un argument. Il n'est tenu pour un qualificatif que si le texte jusqu'au signe égal ne porte pas de barre à lui.
- `main` remet les qualificatifs seuls aux services et garde le paramètre de côté. C'est ce qui préserve la liberté d'ordre du DCL, que les services n'offrent pas.

Aucun verbe ne déclare de paramètre, et à dessein : s'il en déclarait un, `batch submit /NAME=x` remettrait `/NAME=x` à P1.

`SET ENTRY` est le seul verbe en deux mots. Les services CLI tiennent une table de sous-verbes — `ENTRY` est un `CLI_VERB` sous `SET`, avec la table des qualificatifs de la commande — et attendent le second mot tout de suite après le premier ; `main` y remet donc le premier argument qui n'est pas un qualificatif avant de passer le vecteur. Le sous-verbe est réglé comme le verbe, par l'adresse de son descripteur. `/HOLD`, `/KEEP`, `/RESTART` et `/AFTER` admettent la négation pour lui ; `CLI$GET_VALUE` rapporte un qualificatif nié comme présent, aussi ces quatre sont-ils lus par `CLI$PRESENT`.

`s_batch$get_quals` tire les valeurs de la ligne de commande analysée. Ce qui y reste est ce qui est propre à cette commande : découper `/PARAMETERS`, résoudre `/LOG_FILE`, convertir `/AFTER`. La correspondance, l'abréviation, l'ambiguïté et la validation de type appartiennent à la bibliothèque.

Le verbe est pris dans le contexte d'analyse par l'adresse de son descripteur, non en comparant le mot écrit. Les services acceptent toute abréviation non ambiguë ; une seconde résolution faite ici devrait répéter leur règle pour s'accorder avec eux — et ne s'accordait pas : toute abréviation tombait dans la dernière branche de la chaîne, et `BATCH DEL 1` libérait l'entrée qu'il devait supprimer.

3.8 Le paquet StarLet

UTILITY$ROUTINES s'installe en paquet `StarLet` et est exigé comme tel ; aucune copie n'en est portée dans cet arbre. Une seule copie par système mérite qu'on y tienne : les structures de cette bibliothèque sont déclarées dans ses en-têtes et ce sous-système les incorpore — les tables de qualificatifs de `batch.c` sont des `CLI_PQDESC` et la table de messages de `batchmsg.c` un `EMSG_RECORD` — de sorte qu'une seconde révision dans une même image ne s'annoncerait pas.

Ce que le sous-système en emploie :

De	Quoi
`utility_routines.h/.c`	`$PUTMSG_FAO` et le registre des messages, `__util$deflog`, `__util$setlogfd`, `__util$rewindlogfile`, les primitives `__QUEUE`
`starlet.h/.c`	Le moteur FAO, `__util$asctim`, `__util$bintim`, `__util$gettim`, `__util$add_times`
`cli_routines.h/.c`	`__cli$parse`, `__cli$get_value`, `__cli$present` et les tables de qualificatifs
`avproto.h/.c`	Les routines TLV dont se sert le journal de la file

La bibliothèque est liée en partagé là où le paquet en offre une version partagée : une correction qui lui est apportée atteint alors ces images quand la bibliothèque est remplacée, sans attendre qu'elles soient reconstruites. Cela coupe des deux côtés — voir la remarque sur le `SOVERSION` au chapitre 7 — et `BATCH_M_STATICLIB` la fait relier en statique.

Quand une nouvelle révision arrive, remplacez le paquet et passez les tests. `test/queue.c` existe précisément pour cela : un défaut de `$REMQENT` corrigé une fois peut revenir à la révision suivante, et la file des travaux repose dessus. Il a gagné son entretien trois révisions de suite, chacune arrivant avec ce défaut remis en place ; la correction est en amont depuis `V.01-09`.

Une chose encore à surveiller après un changement de révision. La bibliothèque a déplacé ses noms vers un préfixe __ commun, module par module — les services STARLET à une révision, les services CLI à la suivante — de sorte qu'une erreur d'édition de liens nommant `cli$parse` ou `util$asctim` désigne le préfixe, non une routine manquante.

Deux corrections de notre fait ont dû être reportées de révision en révision avant d'être reprises en amont ; toutes deux méritent d'être retenues, car aucune ne se manifestait par une erreur de compilation. L'une passait une valeur par adresse là où l'appelé était passé au passage par valeur ; l'autre rendait un état dont la sévérité avait été ramenée à un nombre impair, ce qui, sous la convention `1 & status`, se lit comme une réussite. La bibliothèque porte désormais ses propres tests de régression, et ils sont passés avec les nôtres.

Chapitre 4

Les structures de données

4.1 BATCH$JOB

Le descripteur de travail, dans `batchdef.h`. Plat, de taille fixe, groupé par type de sorte qu'aucun trou d'alignement n'apparaisse. Il est posé sur le socket exactement tel qu'il est.

Champ	Objet
`jb_entry`	Numéro d'entrée, unique et croissant
`jb_state`	L'un des `JOB$K_*`
`jb_prio`, `jb_retain`, `jb_restarts`	`/PRIORITY`, `/RETAIN`, compte des redémarrages
`jb_pid`, `jb_uid`, `jb_gid`	Le processus et l'identité sous laquelle il s'exécute
`jb_exit`, `jb_signal`	Comment il s'est terminé
`jb_keeplog`, `jb_restart`, `jb_logcwd`, `jb_logclient`	`/KEEP`, `/RESTART`, où va le journal et qui l'a nommé
`jb_qtime`, `jb_after`, `jb_stime`, `jb_etime`	Mis en file, pas avant, commencé, fini
`jb_dev`, `jb_ino`, `jb_size`, `jb_mtime`	Le script tel qu'il était à la soumission
`jb_cpus`, `jb_queue`, `jb_name`, `jb_user`	`/CPU`, `/QUEUE`, `/NAME`, `/USER`
`jb_script`, `jb_logfile`, `jb_cwd`	Chemins absolus
`jb_parm`	`/PARAMETERS`, au nombre de huit

Les champs qu'on ne peut confier à un client — `jb_entry`, `jb_uid`, `jb_gid`, `jb_pid`, les horodatages, l'identification du script — sont posés par `s_batchd$do_submit` et là seulement, quoi que le client ait envoyé.

4.2 BATCHD$JOBENT et BATCHD$CONN

```
typedef struct batchd_jobent_t {
    ENTRY       je_links;   /* en tête : les macros $INSQ/$REMQ l'exigent */
    BATCH$JOB   je_job;
} BATCHD$JOBENT;
```

Les chaînages sont dans une enveloppe et non dans le descripteur. C'est ce qui garde le format du journal indépendant d'eux : ce qui est écrit est `je_job`, et la file pourrait être remplacée demain sans toucher au fichier.

`BATCHD$CONN` tient ce qu'une connexion demande entre les pas d'un transfert : l'en-tête de la requête, le tampon, ce qui en a été déplacé, l'état, l'identité de l'expéditeur et l'échéance. Elle existe parce que le transfert n'est pas un seul appel bloquant.

4.3 Le protocole

`BATCH$IPCHDR` — signature, fonction, valeur de condition de la réponse, longueur de la charge, nombre d'enregistrements, numéro d'entrée — suivi de la charge. `SUBMIT` porte un `BATCH$JOB` ; `SHOW` porte un `BATCH$SHOWREQ` ; `DELETE` et `RELEASE` ne portent rien, le numéro d'entrée étant un champ de l'en-tête ; `SET ENTRY` porte un `BATCH$SETENT`, dont le masque nomme les attributs qu'il règle, de sorte qu'un attribut non nommé n'est pas touché ; `IPC$K_MGRSTAT` ne porte rien à l'aller et un `BATCH$MGRSTAT` au retour.

La réponse à `SHOW` est un en-tête suivi de `mh_count` descripteurs. Toute autre réponse est un en-tête seul. Le compteur des numéros d'entrée voyage avec la réponse à `SHOW` : c'est ce qui permet de distinguer une entrée qui a été et n'est plus d'une qui n'a jamais été délivrée.

Une valeur de condition produite par le contrôleur parvient à l'utilitaire dans `mh_sts`. Les *arguments* de son message, eux, ne voyagent pas ; l'utilitaire nomme donc la condition — la partie du texte qui précède la première virgule, laquelle ne porte aucune conversion — et le texte complet reste dans le journal du contrôleur, où il a été émis en entier.

4.4 Le journal de la file

`BATCHD$JRNHDR` — signature, format du journal, nombre d'enregistrements, compteur des numéros d'entrée — suivi d'autant d'enregistrements.

Un enregistrement n'est **pas** l'image de `BATCH$JOB`. C'est une suite de valeurs étiquetées, écrite par les routines TLV de la bibliothèque, et l'en-tête énonce le format du journal, non la taille du descripteur. Un lecteur prend les étiquettes qu'il connaît, passe outre celles qu'il ne connaît pas, et laisse à sa valeur par défaut un champ dont l'étiquette est absente. Le journal franchit donc un changement de révision dans les deux sens : un BATCH plus récent lit un journal ancien, et un BATCH ancien lit un journal récent autant qu'il l'entend.

Les étiquettes, et les trois règles qui maintiennent cette propriété — une étiquette n'est jamais réattribuée, ne change jamais de type, et un champ nouveau doit avoir une valeur par défaut sous laquelle un ancien journal décrit encore un travail sensé — sont dans `batchjrn.h`. Ajouter un champ à `BATCH$JOB` revient désormais à ajouter une étiquette à côté, et rien de plus.

La longueur précède chaque enregistrement, de sorte qu'un lecteur qui n'en comprend pas un puisse l'enjamber et poursuivre : un enregistrement gâté sur cent est la perte d'un travail, tandis que s'arrêter là serait la perte des quatre-vingt-dix-neuf autres.

L'écriture va dans `journal.tmp`, renommé par-dessus `journal.dat`. Le renommage est atomique au sein d'un système de fichiers ; le journal n'est donc jamais vu à demi écrit. `fsync` précède le renommage, ce qui rend la durabilité réelle et l'écriture coûteuse : voir l'annexe A.

Chapitre 5

Les conventions

Les sources suivent le style OpenVMS de bout en bout. Ce n’est pas un ornement : le lecteur qui connaît la convention reconnaît à vue la durée de vie et l’origine d’un nom.

5.1 La nomenclature

Forme	Sens
`g_nom`	Global
`l_nom`	Local
`s_nom`	Statique — variable, ou routine non exportée de son module
`a_nom`	Paramètre de routine
`BATCH_K_*`	Constante
`BATCH_M_*`	Indicateur de construction
`BATCH$_*`	Valeur de condition
`$MACRO`	Macro
`s_batchd$nom`	Routine statique : le préfixe `s_`, puis le module, puis le nom

Une structure se déclare avec l'étiquette portant le suffixe `_t` et le nom de type portant le séparateur :

```
typedef struct batch_job_t { ... } BATCH$JOB;
```

Un nom dans un fichier CMake ne porte pas de dollar du tout. La nomenclature des sources en met un dans chaque nom, et CMake n'est pas le préprocesseur C : un dollar au sein d'un identificateur n'y appartient pas à celui-ci. Dans un script engendré — `cmake_install.cmake`, `uninstall.cmake` — le défaut est pire qu'une question de style : `cmake_install.cmake` et `uninstall.cmake` sont analysés au moment de l'installation, où `${NOM$K_CHOSE}` n'est pas une référence de variable pour CMake : sous la politique en vigueur, cela vaut un avertissement et une expansion vide, non un refus — et le défaut se montre alors sous les traits d'un service qui n'a pas démarré.

5.2 La mise en forme

Whitesmiths : l'accolade d'un bloc imbriqué se place à l'indentation du corps qu'elle ouvre, celle d'une routine à la colonne zéro. Tabulations seules, de largeur huit ; texte jusqu'à la colonne 132. Déclarations en tête de routine, sans indentation, groupées par type. Une instruction seule sous `if` ne prend pas d'accolades. Un compteur de boucle se déclare dans le `for` là où il le peut.

Chaque module s'ouvre par `MODULE`, `IDENT`, `REV`, `FACILITY`, `ABSTRACT`, `AUTHOR`, `CREATION DATE` et un historique des modifications, et chaque routine par un bloc `FUNCTIONAL DESCRIPTION` disant ce qu'elle fait, ses paramètres d'entrée et de sortie et ce qu'elle rend.

`IDENT` et `REV` paraissent deux fois dans un module — dans l'en-tête et dans les macros `__IDENT__` et `__REV__` — et doivent s'accorder.

L'identification est `X-aa-bb`, à la manière VMS, et `REV` s'en déduit partie pour partie : `X-aa-bbECOcc` donne `aa.bb.cc`, ainsi `X-01-29` donne `01.29.00`. Une petite correction de faute n'élève pas `bb` ; elle se marque `ECO`, et le numéro de l'ECO prend la troisième place — `X-01-29ECO01` donne `01.29.01`. Chaque entrée de l'historique des modifications porte `RRL` entre la date et l'identification.

5.3 La convention d’appel

Une routine rend une valeur de condition et livre son résultat par un paramètre de sortie.

```
if ( $IS_FAILURE(s_batchd$job_by_entry(a_entry, &l_ent)) )
    return  $PUTMSG_FAO(BATCH$_NOSUCHENT, a_entry);
```

Les exceptions sont au nombre de trois, et chacune le dit dans son propre commentaire : `s_batchd$sighandler`, dont la signature est imposée par le système, et les deux routines `main`.

`$PUTMSG_FAO` rend la condition qu’on lui a donnée, ce qui permet de loger un diagnostic et un retour dans une seule expression.

La réussite s’éprouve par `$IS_SUCCESS` ; une cause précise se distingue par `$MATCH_COND`, qui compare l’identification et néglige la sévérité.

5.4 Les diagnostics

Tout diagnostic passe par `$PUTMSG_FAO`. Le texte est dans la table, non au point d'appel ; là où une condition sert plusieurs endroits, la partie distinctive est un paramètre du texte plutôt qu'une condition à part.

L'échec d'une routine de la bibliothèque C se rapporte là où il se produit, avec le nom de la routine, son résultat et `errno` :

```
if ( 0 > (l_rc = mkdir(l_work, a_mode)) && (errno != EEXIST) )
    return  $PUTMSG_FAO(BATCH$_MKDIRERR, l_work, l_rc, errno, strerror(errno));
```

Rendre une condition vers le haut en laissant derrière soi le compte rendu de la bibliothèque d'exécution perd la seule information qui aurait identifié l'échec.

5.5 La mise en forme des sorties

Rien dans le sous-système n'appelle `printf` sous quelque forme que ce soit, et aucun module n'inclut `<stdio.h>`. La sortie passe par le moteur FAO — `util$put` pour une ligne à l'utilisateur, `util$mkfspec` pour une chaîne — dans les directives d'OpenVMS :

Directive	Effet
`!AZ`	Chaîne terminée par zéro
`!AC`	Chaîne comptée, un conteneur `ASC`
`!UL`, `!SL`	Décimal non signé et signé
`!nZL`	Décimal complété de zéros, de largeur n
`!n<...!>`	Champ de largeur fixe, complété d'espaces, tronqué
`!n*c`	Caractère répété n fois
`!%S`	`s` lorsque le dernier nombre converti n'est pas 1

La directive `!/` n'est pas employée. Elle donne le couple retour chariot et saut de ligne d'OpenVMS, qu'un flux Unix n'attend pas ; un séparateur de ligne s'écrit en `\n` littéral.

Ce que le sous-système dit de lui-même passe par une condition ; ce qu'il a été lancé pour produire — le tableau des entrées, l'analyse du journal — reste du texte simple, car ce sont des données que l'on lit et que l'on découpe.

Chapitre 6

Faire une modification

6.1 Ajouter un qualificatif

Soit le qualificatif `/NICE=n` de l'utilitaire.

1. `batchdef.h` — ajoutez le champ à `BATCH$JOB`, dans le groupe de son type.
2. `batchjrn.h` — donnez-lui une étiquette, et `util.c` l'encode et le décode. Voyez les trois règles de la section 4.4 : le journal doit rester lisible dans les deux sens.
3. `batch.c` — ajoutez l'énumérateur aux `QUAL$K_*` et la ligne à `s_quals`, avec le type que les services CLI doivent valider : `CLI$K_NUM`, `CLI$K_QSTRING`, `CLI$K_OPT` pour un qualificatif sans valeur, `CLI$K_KWD` avec une liste `CLI_KEYWORD` pour un ensemble de mots. **L'ordre de l'énumération et celui de la table doivent s'accorder** : ils s'indexent l'un l'autre.
4. `batch.c` — ajoutez le champ à `BATCH$CLICTX`, tirez la valeur dans `s_batch$get_quals`, posez-la dans le descripteur dans `s_batch$do_submit`.
5. `batch.c` — ajoutez-le au texte d'usage de `s_batch$usage` et à l'`ABSTRACT` du module. Comptez les conversions : le texte d'usage en nomme autant qu'il reçoit d'arguments, et le compilateur ne le vérifie pas.
6. `batchd.c` — agissez en conséquence, d'ordinaire dans `s_batchd$job_start`.
7. `batch.c` — affichez-le dans `s_batch$show_full` s'il mérite d'être vu.
8. Les bibliothèques d'aide, dans les trois langues, et le *Guide de l'utilisateur*.
9. `test/smoke.sh` — une vérification qu'il est accepté et qu'une valeur mal formée est refusée.

Prenez garde aux abréviations : un qualificatif nouveau partageant un début avec un existant allonge la plus courte abréviation non ambiguë des deux, et cela se voit de l'utilisateur.

6.2 Ajouter une condition

1. `batchmsg.h` — prenez le prochain numéro libre **dans le bon domaine**, et donnez-lui la sévérité qui décrit l'issue, non l'humeur de l'appelant. Deux conditions ne peuvent partager un numéro ; une vérification du test s'en assure.
2. `batchmsg.c` — ajoutez la ligne à `s_msgtab`, en directives FAO et sans enveloppe propre. Le nombre de directives du texte doit égaler le nombre d'arguments que passe chaque point d'appel ; le compilateur ne le vérifie pas, le format venant de la table à l'exécution. `test/smoke.sh` vérifie qu'aucun texte ne porte de conversion printf et que tout diagnostic passe par `$PUTMSG_FAO`.
3. Le *Guide de l'utilisateur*, chapitre 6 — la mnémonique, l'explication et la conduite à tenir.

6.3 Ajouter une commande

1. `batchipc.h` — un code de fonction `IPC$K_*`, et une structure de charge utile si la commande en porte une.
2. `batch.c` — un verbe dans `s_verbs`, un énumérateur dans `VERB$K_*` **dans le même ordre**, un traitant à côté de `s_batch$do_submit`, et une branche du `switch` de `main`.
3. `batchd.c` — un `case` dans `s_batchd$conn_dispatch` et un traitant à côté de `s_batchd$do_submit`. **Assemblez la réponse en mémoire** ; n'écrivez pas au descripteur depuis le traitant.
4. Posez `*a_dirty` si la commande a changé la file, afin que la boucle réordonnance et journalise.
5. Les bibliothèques d'aide et le *Guide de l'utilisateur*.

6.4 Modifier le descripteur de travail

Le journal se décrivant lui-même, un champ nouveau ne rend plus un journal existant illisible : il suffit de lui donner une étiquette et une valeur par défaut sensée. Ce qui demeure exigé, ce sont deux invariants :

- le descripteur reste plat et de taille fixe — pas de pointeurs, rien qui n'ait de sens que dans l'espace d'adressage qui l'a écrit ;
- les champs qu'un client ne peut poser restent posés par `s_batchd$do_submit` seul.

6.5 Ajouter une vérification au test

`test/smoke.sh` lève un contrôleur sur une racine temporaire et l'exerce par l'utilitaire. Une vérification est trois choses : une action, une attente de l'état attendu, et `l_check`.

```
"${g_batch}" submit "${g_root}/scripts/ok.sh" /NAME=probe >/dev/null 2>&1
l_wait_for "probe" show /all
l_check $? "the job reached the queue"
```

Attendez un état ; ne dormez pas un moment. La durée d'un travail dépend de la charge de la machine et n'est pas connue d'avance. `l_wait_for` et `l_wait_gone` interrogent pendant dix secondes.

Une vérification tardive a besoin d'un contrôleur à elle. Arrivé au bas du fichier, les places d'exécution du contrôleur principal sont prises par les travaux longs des vérifications précédentes ; une vérification qui a besoin qu'un travail s'exécute lève donc son propre contrôleur sur une racine à part.

Le test se nettoie après lui : les travaux qu'il a lancés sont signalés par groupe de processus, un travail survivant au contrôleur et certains étant orphelins à dessein du fait de la vérification du redémarrage.

Chapitre 7

Le système de construction

7.1 Disposition

Fichier	Objet
`CMakeLists.txt`	L'identification, les options de compilation communes, l'en-tête engendré, les paquets StarLet et HELP, la cible de retrait, la cible `dist`
`src/CMakeLists.txt`	Les cibles et l'installation
`test/CMakeLists.txt`	L'enregistrement des tests
`etc/batchd.service.in`	Le modèle de l'unité systemd
`etc/batchd.init.in`	Le modèle du script de démarrage System V, pour un système sans systemd
`cmake/uninstall.cmake.in`	Le script de retrait
`src/batchvers.h.in`	Le modèle de l'en-tête d'identification

`batchcom` est une bibliothèque d'objets portant `util.c` et `batchmsg.c` ; les deux images la lient, de sorte que ni l'une ni l'autre n'est compilée deux fois et que toutes deux sont bâties sur un seul et même objet.

Deux cibles se tiennent à part des images. `doc_pdf` réengendre les manuels depuis leur markdown, et est bâtie avec le reste quand `BATCH_M_PDF` est posé ; elle écrit dans `doc/`, à côté du markdown, plutôt que dans l'arbre de construction — les manuels rendus appartiennent aux sources et sont tenus en gestion de versions avec elles, de sorte que lire un manuel ne demande aucune installation de TeX et que le sous-système se remet d'un seul tenant. `dist` assemble le kit et le nomme d'après l'installation et la révision qu'il porte, `BATCH-X-01-07.tar.gz`, à la manière des kits des produits voisins.

La bibliothèque d'aide est engendrée par langue : `doc/batch_help<suffixe>.md` devient `doc/batch<suffixe>.hlb`, et le sous-répertoire de locale où elle s'installe est nommé à côté. Une quatrième langue coûte une ligne de la liste.

7.2 Les options de compilation

Elles sont tenues dans une cible d’interface, `batch_options`, plutôt que dans le `CMAKE_C_FLAGS` global, afin qu’une cible puisse être ajoutée sans les répéter :

```
-Wall -Wextra -Wno-unused-parameter -Wformat=2 -Wshadow -Wpointer-arith -Wwrite-strings
```

`BATCH_M_STRICT=ON` y ajoute `-Werror`.

La norme est gnu11 : le signe dollar dans un identificateur est une extension GNU, et toute la convention de nomenclature repose dessus.

Sur la liaison partagée. UTILITY$ROUTINES est liée en partagé là où le paquet en offre une version partagée. Cela coupe des deux côtés : les dispositions de ses structures sont déclarées dans ses en-têtes et ce sous-système les incorpore, de sorte qu’une révision qui change l’une d’elles doit élever son `SOVERSION`, faute de quoi les images liront la bibliothèque nouvelle selon l’ancienne disposition. `BATCH_M_STATICLIB=ON` la relie en statique — pour une image de secours, ou là où cette discipline ne peut être tenue pour acquise.

7.3 Deux maladresses

`configure_file` substitue `@NOM@` mais ne reconnaît pas un dollar au sein d’un nom de variable. Les réglages voyagent donc vers les modèles par des noms auxiliaires sans le séparateur — `BATCH_CFG_SPOOLROOT` porte `BATCH_K_SPOOLROOT` — et les macros qu’on en tire le retrouvent.

Tout ce qui est destiné à un chemin d’installation doit être calculé **avant** `configure_file` et avant que la cible qui s’en sert ne soit définie. Un `$<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>` évalué trop tôt donne un chemin vide, et l’erreur est silencieuse : ni la construction ni l’installation ne s’en plaignent, seul s’en plaint celui qui se sert du paquet.

Chapitre 8

Les tests

8.1 Ce qu’il y a

Cible	Ce qu’elle exerce
Les tests du paquet StarLet	Passés avec les nôtres quand la bibliothèque est bâtie avec le sous-système
`batch_queue`	Les primitives de file de cette bibliothèque : retrait en tête, en queue, au milieu et de la dernière entrée, les chaînages parcourus dans les deux sens après chacun
`batch_smoke`	Le sous-système de bout en bout, par l’utilitaire, sur une racine temporaire

```
ctest --test-dir build --output-on-failure
```

Un test échoue quand sa sortie correspond à `%BATCH-[EF]-` ; une vérification rapporte donc sa propre issue et `ctest` n’a rien à en savoir.

8.2 Ce que le test de bon fonctionnement garde

Chaque vérification existe parce que quelque chose a été un jour de travers, ou pourrait plausiblement l’être sans qu’on le remarque :

Vérification	Ce qu’elle garde
Travail conservé sur échec, retiré sur réussite	La règle de conservation, qui est ce qui rend `SHOW` utile
`/HOLD` et `/AFTER`	L’état de retenue et sa libération
Limite de travaux	L’ordonnanceur
`DELETE`, et une liste d’entrées	Le retrait d’une entrée, le signalement d’un travail en cours, et qu’aucun élément de la liste ne soit passé sous silence
Redémarrage du contrôleur	Le journal, et qu’un successeur n’essuie pas de refus tant que le prédécesseur tient encore son socket
Chemin absolu pris pour un paramètre	La distinction entre un qualificatif et un chemin
Formes relatives `+3`, `+:30`, `+::45`	Un défaut de l’analyseur que la seule forme exercée auparavant n’atteignait pas
Verbe abrégé	Que `DEL` supprime au lieu de libérer
Heure locale des deux côtés	Qu’une heure entrée puis affichée revienne inchangée
Étiquette inconnue dans le journal	Que le journal franchisse une révision dans les deux sens
Nom de travail qui sortirait du répertoire des journaux	Le seul fichier que le contrôleur ouvre avec ses privilèges
Journal nommé par un client	Qu’il ne soit ni créé, ni tronqué, ni retiré par le privilégié
Script remplacé après la soumission	Que ce ne soit pas un autre fichier qui s’exécute
Numéros de message	Que deux conditions n’en partagent pas un
`/USER`	Que seul le privilégié puisse le demander, et que le journal en garde trace
Un client muet	Que les connexions soient servies sans blocage
Un travail qui ne lance rien survivant à un redémarrage	Une indication de changement qui était jetée

Les dernières lignes méritent d’être lues deux fois : ce sont des vérifications écrites *après* qu’un défaut a atteint une construction, ce qui est la seule manière de savoir qu’une vérification vaut d’exister.

Annexe A

Limitations connues

Chacune est une décision qui pourrait être revue, et chacune est donnée avec l'endroit où elle vit.

Le journal est écrit en entier et vidé sur disque à chaque changement. `s_batchd$journal_save`. Mesuré sur une file de 300 entrées en ext4 : 4579 µs pour une opération qui écrit le journal contre 29 µs pour une qui ne l'écrit pas, dont quelque 3700 µs de vidage. Trois sorties, par charge de travail croissante : grouper les écritures sur un court intervalle, faire du vidage une affaire de politique, ou écrire le journal par incréments et le compacter de temps à autre. La forme étiquetée a déjà beaucoup réduit ce qui est écrit — un paramètre vide n'est plus écrit du tout.

Une seule file. Le descripteur porte `jb_queue` et `SHOW` filtre dessus, mais la limite de travaux est commune à tout le sous-système. Une seconde file demanderait une limite par file et un ordonnanceur qui les connaisse : `s_batchd$schedule`.

Ni la courtoisie, ni les limites de ressources, ni le groupe de contrôle ne se posent par travail. L'affinité se pose, depuis `/CPU` et `/LCORE` ; le reste est hérité du contrôleur. `s_batchd$job_start` est l'endroit où l'un ou l'autre viendrait.

Le contrôleur s'éveille une fois par seconde au repos. `BATCHD$K_POLLMS`, pour l'amour de `/AFTER`. Calculer l'échéance la plus proche supprimerait ces réveils sans changer le comportement.

Les diagnostics de la ligne de commande paraissent sous `%CLI_RTNS-` plutôt que sous `%BATCH-`, la bibliothèque portant son propre nom d'installation. Laissé tel quel à dessein : le lecteur distingue une plainte sur la forme d'une commande d'une plainte sur son sens.

Il subsiste une fenêtre étroite entre la dernière vérification du script et `execv`. Le script est épinglé par périphérique, inœud, taille et date à la nanoseconde, et vérifié sous l'identité qui l'exécutera ; mais entre cette vérification et l'`execv` le chemin pourrait encore désigner autre chose. Cela ne donne plus d'élévation de privilège — l'exécution se fait sous l'uid du client — et c'est dit ici par souci d'exactitude.